FUSION LAYER TECHNICAL REPORT

1. Executive Summary

This report documents the current architecture and implementation status of the fusion layer for the Cyber Analytics multi-model email threat system. The fusion layer is the terminal decision component that combines model probabilities from the header, body, and malware branches into one final threat score and classification output. The implementation is intentionally late-fusion and production-oriented for Phase 1, emphasizing modularity, explainability, robust handling of partial modality availability, and operational compatibility with cloud event-driven workflows.

The fusion layer currently supports two fusion strategies: logistic regression stacking (primary) and soft voting (baseline). It enforces strict input and output contracts, validates data quality, handles missing malware probabilities for emails without attachments, maps final scores to project-defined risk levels, and returns an explainable output record that includes both the fusion method and the set of modalities used.

2. System Context and Scope

The broader platform architecture is organized as a staged pipeline:

1. Email ingestion (Gmail/API)
2. Raw object storage in S3
3. Parsing and preprocessing
4. Specialized branch models (header, body, malware)
5. Fusion layer scoring
6. Final SOC-facing output (dashboard/API/alert path)

This report focuses specifically on stage 5. Upstream ingestion, parsing, and branch-model training are out of scope for this module and are owned by adjacent contributors. The fusion layer is intentionally decoupled so that upstream model updates do not require redesign of the downstream decision component.

3. Architectural Design

3.1 Late Fusion Design Choice

The architecture uses late fusion rather than end-to-end neural fusion for Phase 1. This choice was made for practical and operational reasons:

- lower integration complexity across heterogeneous modalities
- easier debugging and failure isolation
- clear interpretability and explainability
- clean support for missing branch outputs
- faster implementation and validation under project timeline constraints

3.2 Component Breakdown

The fusion module is organized into explicitly separated concerns:

- ‘contracts.py‘: strict row-level schema contracts
- ‘validators.py‘: dataframe-level quality and rule enforcement
- ‘preprocess.py‘: modality-availability annotation and feature derivation
- ‘soft_voting.py‘: baseline fusion engine
- ‘logistic_fusion.py‘: trained stacking fusion engine
- ‘risk_mapping.py‘: score-to-risk conversion logic
- ‘loaders.py‘: local CSV loading/writing and in-memory dataframe validation
- ‘s3_io.py‘: cloud I/O helpers for S3 URIs and object read/write
- ‘lambda_handler.py‘: event-driven Lambda runtime entrypoint

This decomposition enables unit-level testing, explainable behavior, and maintainable ownership boundaries.

4. Data Contracts and Decision Output

4.1 Inference Input Contract

Per-email inference records must follow:

‘email_id, p_header, p_body, p_malware‘

Where each probability is in ‘[0,1]‘ when present, and at least one modality must exist. ‘p_malware‘ can be null when an email has no attachment.

4.2 Training Input Contract

Training records add a binary label:

‘email_id, p_header, p_body, p_malware, true_label‘

4.3 Output Contract

Fusion outputs follow:

‘email_id, final_score, final_label, risk_level, models_used, fusion_method‘

Where:

- ‘final_score‘: fused probability in ‘[0,1]‘
- ‘final_label‘: binary classification using configured threshold
- ‘risk_level‘: mapped category based on score band
- ‘models_used‘: present modalities (‘header|body|malware‘, etc.)
- ‘fusion_method‘: ‘soft_voting‘ or ‘logistic_regression_stacking‘

5. Fusion Methods

5.1 Soft Voting (Baseline)

Soft voting computes the arithmetic mean of available modality probabilities only. Missing values are ignored and do not contribute synthetic values to the average.

5.2 Logistic Regression Stacking (Primary)

The stacking model is trained on:

- filled probabilities: ‘p_header_filled‘, ‘p_body_filled‘, ‘p_malware_filled‘
- presence flags: ‘has_header‘, ‘has_body‘, ‘has_malware‘
- modality count: ‘models_present_count‘

This feature design preserves modality-availability information while keeping inference stable when values are missing.

6. Missing-Modality Strategy

The model is explicitly designed not to fail when one branch is absent.

- Soft voting: excludes missing modalities from mean calculation.
- Logistic stacking: fills missing probabilities with neutral imputation (‘0.50‘ default) while preserving true absence via binary indicators.

This ensures robust handling of common operational cases such as missing malware scores for non-attachment emails.

7. Risk Mapping and Explainability

Final score bands are mapped as follows:

- ‘0.00 <= score < 0.30‘: low / benign
- ‘0.30 <= score < 0.70‘: medium / suspicious
- ‘0.70 <= score <= 1.00‘: high / malicious

Explainability is supported through:

- explicit contract validation errors
- interpretable logistic coefficients saved in metadata
- transparent modality trace via ‘models_used‘
- explicit fusion method labeling on each output row

8. Lambda + S3 Runtime Integration

The cloud runtime is now implemented using ‘lambda_handler.py‘ and ‘s3_io.py‘.

The Lambda handler supports:

1. direct invocation payloads with explicit ‘input_s3_uri‘
2. native S3 event payloads (‘Records[0].s3‘)

Runtime sequence:

1. resolve config and fusion method
2. resolve input object URI
3. load inference CSV from S3
4. validate and normalize input
5. execute selected fusion method
6. write prediction CSV to S3
7. return execution metadata (‘rows_scored‘, output URI, method)

Required IAM access includes S3 read on input/config/artifact locations and S3 write on output prefix.

9. Verification Status

Current automated validation status: **21 tests passing**.

Coverage includes:

- contracts and validators
- preprocessing and missing-modality behavior
- soft voting and logistic fusion behavior
- model save/load and metadata integrity
- risk mapping boundaries
- S3 utility parsing/read/write behavior
- Lambda handler S3-event and payload flow

This indicates the fusion module is ready for upstream integration and cloud deployment wiring.

10. Current Readiness and Next Steps

The fusion layer is technically complete for Phase 1 objectives and operationally ready to ingest real branch-model outputs. Remaining effort is primarily integration and orchestration:

1. connect real branch probability outputs into fusion input records
2. enable production trigger chain (S3 event, EventBridge, or Step Functions)
3. validate full end-to-end behavior on real project data
4. define downstream alerting and dashboard consumption path

11. Conclusion

The implemented fusion layer provides a robust, explainable, and cloud-compatible decision stage for the multi-model cyber-threat pipeline. It satisfies the required late-fusion methodology, supports missing modalities, and produces SOC-friendly output artifacts. With Lambda + S3 runtime support now in place, the module is positioned for direct integration with upstream branch model outputs and final platform orchestration.